Snake Game RL

Gianluca Caregnato [1]

Abstract

This report presents a comparative study of three reinforcement learning algorithms applied to the Snake game: Deep Q-Network (DQN), REINFORCE with baseline, and Actor-Critic. A Greedy BFS heuristic serves as a non-learning baseline. All agents are evaluated in both fully and partially observable settings, with action masking to prevent wall collisions. DQN achieves the best performance, exceeding the baseline by 22%.

1. Introduction

In this work, I approach the problem from multiple angles. I first establish a non-learning baseline using a **Greedy Breadth-First Search (BFS)** heuristic, which deterministically follows the shortest path to the fruit. This baseline provides a principled reference point: it achieves zero wall hits and a consistent average reward, but remains fundamentally limited because it is purely reactive and does not reason about the long-term consequences of the snake's growing body.

I then implement and compare three reinforcement learning algorithms of increasing sophistication. **Deep Q-Network (DQN)** approximates the action-value function with a neural network and learns from stored experience through off-policy updates. **REINFORCE with baseline** directly optimizes the policy via Monte Carlo estimates of the return, using a learned value function to reduce the variance of the gradient signal. **Actor-Critic**, combines a policy network (actor) with a value network (critic) in a shared architecture, computing advantages from bootstrapped returns rather than full Monte Carlo rollouts, which results in lower-variance updates at the cost of introducing some bias.

All three agents share a common convolutional neural network backbone tailored to the grid structure of the state, and all benefit from *action masking* a technique that prevents the agent from selecting actions that would lead into a wall, effectively pruning clearly suboptimal choices from the action space.

A key aspect of this study is the comparison between two environment settings: **fully observable**, where the agent sees the entire 7×7 board, and **partially observable**, where it only receives a local 5×5 window centered on the snake's head. This comparison reveals how limited information impacts both the heuristic baseline and the learned agents, and highlights the ability of RL methods to generalize under uncertainty.

2. Environment

The game takes place on a 7×7 grid with walls on the border, leaving a 5×5 playable area containing one fruit and a snake (head + body). The environment runs N boards in parallel (N=1000 for training, N=100 for evaluation). Each cell is one-hot encoded into 4 channels (EMPTY, FRUIT, BODY, HEAD), and the agent chooses among four actions: Up, Right, Down, Left.

Rewards. Eating a fruit yields $+0.5$ and grows the snake by one segment. Hitting a wall gives -0.1 (the snake stays in place). Self-collision gives -0.2 and truncates the body from the collision point to the tail, the penalty is fixed regardless of how many segments are lost. Normal steps yield 0. Filling the entire board awards $+1.0$ and resets it, though this event is rare.

Continuous environment. The game **never terminates**: there are no episode boundaries. This is the environment's most critical property for algorithm design. DQN handles this naturally through single-step bootstrapping. REINFORCE, however, requires an adaptation: since it is a Monte Carlo method that assumes complete episodes, I collect fixed-length rollouts (50 steps) and bootstrap $V(s_T)$ at the end to approximate the infinite-horizon return (Section 4.3). Actor-Critic uses n-step returns with bootstrapping by design (n=5), so no modification is needed.

Observability. I evaluate two settings: **fully observable**, where the agent sees the entire 7×7 board, and **partially observable**, where it receives only a 5×5 window centered on the head. In the latter, the fruit may be invisible, making

[1]University of Padua, Italy. Correspondence to: Gianluca Caregnato <gianluca.caregnato@studenti.unipd.it>.

Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021. Copyright 2021 by the author(s).

the task considerably harder.

3. Baseline: Greedy BFS Heuristic

As a non-learning reference, I implement a greedy heuristic that uses **Breadth-First Search** (BFS) to find the shortest path from the snake's head to the fruit, avoiding walls and body segments. If the fruit is reachable, the agent takes the first action along the shortest path; otherwise, it selects a random safe action (one that does not collide with a wall).

Action masking. All agents, including the baseline, benefit from *action masking*: before selecting an action, I check which of the four moves would immediately collide with a wall and exclude them. For neural network agents, this is implemented by setting the logits of invalid actions to $-\infty$ before the softmax (policy methods) or the argmax (DQN). This simple technique eliminates all wall hits and narrows the effective action space.

Partial observability adaptation. For the partially observable setting, the baseline only has access to the 5×5 local window around the head. If the fruit is visible within this window, BFS is applied on the local view. If not, the agent takes a random safe action, since it has no information about the fruit's location. This "fair" baseline ensures the comparison with RL agents is meaningful: both the heuristic and the learned agents operate under the same information constraint.

4. Methods

All agents are implemented in **TensorFlow 2 / Keras**, which is the same framework used by the environment.

4.1. Shared CNN Backbone

Since the state is a 2D grid, all agents share a convolutional backbone: two `Conv2D` layers ($4\to32$ and $32\to64$ filters, 3×3 kernels, padding `same`, ReLU activations) followed by a fully connected layer that compresses the flattened feature maps into a 128-dimensional vector. This compact architecture is appropriate for the small board size and avoids overfitting. Each algorithm attaches a different head to this backbone: a Q-value head for DQN, a policy head for REINFORCE, or both a policy and value head for Actor-Critic.

4.2. Deep Q-Network (DQN)

DQN approximates the action-value function $Q(s, a; \theta)$ with the CNN backbone plus a two-layer head ($128\to64\to4$). The agent selects actions via an ε-greedy policy: with probability $\varepsilon = 0.05 + 0.95 \cdot e^{-t/2000}$ it takes a random valid action, otherwise it picks $\arg\max_a Q(s, a)$.

Experience replay. Transitions (s, a, r, s') from all parallel boards are stored in a replay buffer of capacity 500 000. At each training step, a mini-batch of 256 transitions is sampled uniformly at random. This breaks temporal correlations and makes the training data approximately i.i.d., which is essential for stable SGD convergence.

Target network and Double DQN. A target network $Q(s, a; \theta^-)$ provides stable TD targets. It is updated via Polyak averaging ($\theta^- \leftarrow \tau\,\theta + (1{-}\tau)\,\theta^-$, $\tau{=}0.005$) every 200 training steps. I use the **Double DQN** variant to reduce overestimation bias: the online network selects the best action, while the target network evaluates it:

$$y = r + \gamma\, Q\big(s',\, \arg\max_{a'} Q(s', a'; \theta);\, \theta^-\big). \qquad (1)$$

The loss is $\mathcal{L} = \mathrm{MSE}(Q(s, a; \theta),\, y)$, optimized with Adam.

4.3. REINFORCE with Baseline

REINFORCE directly optimizes a stochastic policy $\pi(a|s; \theta)$ via the policy gradient theorem. The policy head outputs a softmax distribution over actions (with invalid actions masked to $-\infty$), and actions are sampled from this distribution during training.

Return computation with bootstrapping. For each training iteration, the agent collects a rollout of $T{=}50$ steps. Since the environment is continuous (Section 2), I cannot set $G_T = 0$ at the rollout boundary. Instead, I bootstrap:

$$G_T = V(s_T; w), \qquad G_t = r_t + \gamma\, G_{t+1} \quad \text{for } t = T{-}1, \ldots, 0, \qquad (2)$$

where $V(s; w)$ is a learned value network (the baseline). This value network has the same CNN backbone but with a scalar output head ($128\to64\to1$).

Policy and value updates. The advantage $A_t = G_t - V(s_t; w)$ serves as a variance-reduced learning signal. The policy is updated by ascending the gradient:

$$\nabla_\theta J \approx \frac{1}{TN} \sum_{t,n} \nabla_\theta \log \pi(a_t^{(n)} | s_t^{(n)}; \theta)\, A_t^{(n)}, \qquad (3)$$

while the value network is trained to minimize $\mathrm{MSE}(V(s_t), G_t)$. I apply return whitening (standardizing returns to zero mean and unit variance) for training stability and add an entropy bonus ($\beta{=}0.01$) to encourage exploration.

4.4. Actor-Critic

The Actor-Critic agent combines a policy (actor) and value function (critic) in a single network with a shared CNN backbone and separate heads. This is more parameter-efficient than REINFORCE, which uses two independent networks.

The key difference from REINFORCE is in the return estimation: instead of 50-step Monte Carlo rollouts, Actor-Critic uses n-step returns (n=5) with bootstrapping:

$$G_t = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n}; w). \quad (4)$$

This introduces some bias compared to Monte Carlo but substantially reduces variance, leading to more stable and data-efficient learning. The total loss combines the policy gradient loss, a value prediction loss weighted by 0.5, and the entropy bonus.

4.5. Hyperparameters

Table 1 summarizes the hyperparameter configuration used for each algorithm.

Parameter	DQN	REINFORCE	A-C
Learning rate	5×10^{-4}	5×10^{-4} / 1.5×10^{-3}	5×10^{-4}
Discount γ	0.95	0.95	0.95
Batch / rollout	256	50 steps	5 steps
ε decay	2 000 steps	—	—
Entropy coeff.	—	0.01	0.01
Training iters	100 000	10 000	10 000
Parallel boards	1 000	200	200

Table 1. Hyperparameter configuration for each algorithm.

5. Experimental Results

All agents are evaluated on 100 parallel boards for 1 000 steps with seed 0, using greedy action selection (no exploration). Table 2 and Figure 1 report the main metrics.

Env	Agent	Avg Rew	Fruits	Self	Wall	Wins
Full	Baseline	0.1124	23 484	2 516	0	2
	DQN	**0.1368**	**29 156**	4 477	0	2
	REINFORCE	0.1308	28 774	6 514	0	0
	A-C	0.1307	28 848	6 777	0	0
Partial	Baseline	0.0878	18 475	2 280	0	1
	DQN	**0.1234**	**26 419**	4 346	0	0
	REINFORCE	0.1173	25 691	5 559	0	0
	A-C	0.1193	26 116	5 646	0	0

Table 2. Evaluation results (100 boards, 1 000 steps, seed 0).

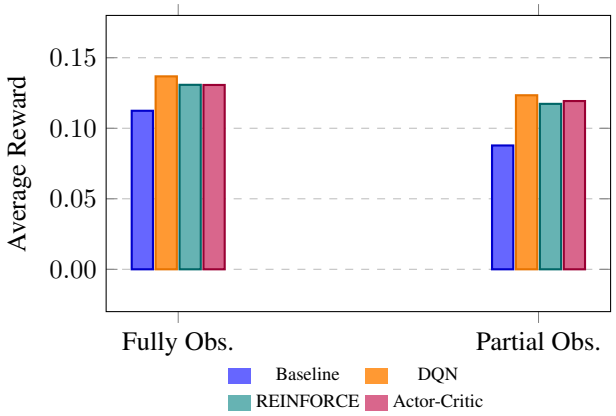

Figure 1. Average reward (100 boards, 1 000 steps, seed 0).

All RL agents surpass the baseline. In the fully observable setting, DQN achieves the highest average reward (0.1368), followed by REINFORCE (0.1308) and Actor-Critic (0.1307). All three exceed the BFS baseline (0.1124) by 16–22%. The advantage of RL agents stems from their ability to learn long-term strategies, the baseline is purely reactive and makes no effort to avoid future self-collisions.

Partial observability degrades all agents. The baseline drops from 0.1124 to 0.0878 (-22%) because it can no longer always locate the fruit. RL agents show a smaller degradation ($\approx 10\%$ for DQN), demonstrating that neural networks learn implicit exploration strategies even with limited vision.

Self-collision trade-off. RL agents eat significantly more fruit than the baseline but also suffer more self-collisions. REINFORCE and Actor-Critic have the highest self-hit rates (6 777), which correlates with their slightly lower rewards. DQN achieves the best balance between fruit collection and self-avoidance, with the fewest self-hits among all RL agents.

Why wins are nearly zero. A "win" requires the snake to fill every cell of the 5×5 playable area, 25 fruits with no self-collisions, and this is an extremely demanding event. As the snake grows, the number of safe paths shrinks combinatorially, and even a single self-collision resets progress by truncating the body. Over 100 boards and 1 000 steps, only the baseline and DQN managed 2 wins in the fully observable setting, while in the partially observable environment only the baseline managed 1 win. This highlights a fundamental difficulty of Snake: the reward for winning ($+1.0$) is rarely seen during training, so the agents instead converge to a policy that maximises steady fruit collection without ever learning the late-game navigation skills needed to fill the board completely.

Learning curves. Figure 2 and Figure 3 show the training reward over iterations. DQN converges slowly over 100 000 iterations. REINFORCE and Actor-Critic reach comparable performance in 10 000 iterations.

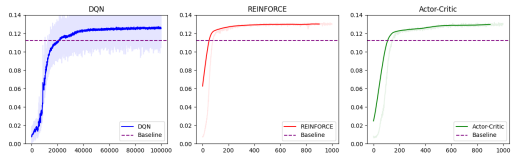

Figure 2. Learning curves for the fully observable environment.

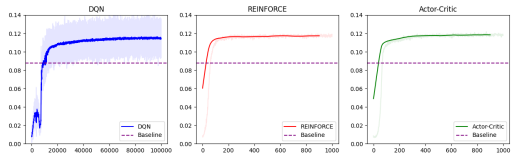

Figure 3. Learning curves for the partially observable environment.

6. How to run it

To reproduce the results, two evaluation scripts are provided.

Running the `snake/evaluate.py` script directly will evaluate only the best performing model (Deep Q-Network) along with the Greedy BFS baseline. The pre-trained weights for the DQN agent are already provided in the `snake/weights` directory.

For a comprehensive evaluation of all models, the Jupyter notebook `snake/evaluate_all.ipynb` is provided. This notebook is already executed and displays all the evaluation metrics and learning curves. If you wish to re-run it yourself, the notebook is configured to automatically download the pre-trained weights for all three RL agents (DQN, REINFORCE, Actor-Critic, totaling ≈ 8.71 MB) into a local folder and perform a full evaluation across all models and environments.

7. Conclusion

The goal of this project was to investigate whether reinforcement learning agents can learn to play Snake better than a hand-crafted heuristic, and to understand how different algorithmic choices affect the outcome. To answer this question, I implemented three algorithms, DQN, REINFORCE with baseline, and Actor-Critic, and compared them against a Greedy BFS baseline in both a fully observable and a partially observable version of the environment.

The results show that all three RL agents consistently outperform the baseline. DQN, in particular, achieves the best trade-off between fruit collection and self-avoidance, reaching an average reward of 0.1368 in the fully observable case, roughly 22% higher than the BFS heuristic. REINFORCE and Actor-Critic reach similar performance levels, though they tend to suffer from more frequent self-collisions, which slightly drags their average reward down. The key advantage of the learned policies over the heuristic is their ability to reason about long-term consequences: the BFS baseline greedily chases the nearest fruit without considering that its trajectory may lead to an unavoidable self-collision a few steps later, while the neural network policies learn, at least partially, to avoid such situations.

An interesting finding is how the three algorithms handle the non-episodic nature of the environment. Because the game never terminates, REINFORCE cannot rely on natural episode boundaries to compute Monte Carlo returns. I addressed this by bootstrapping a value estimate at the end of each rollout, effectively turning REINFORCE into a hybrid between Monte Carlo and temporal-difference methods. Actor-Critic, which already bootstraps by design, required no such adaptation and was more straightforward to implement. DQN, being fully off-policy, was unaffected by the continuous structure entirely.

Partial observability reduces performance across the board, but the RL agents degrade far less than the baseline ($\approx 10\%$ vs. 22%). This suggests that the convolutional network learns some form of implicit memory or exploration strategy that compensates for the missing information, while the BFS baseline simply cannot find a path when the fruit lies outside its 5×5 field of view.

Finally, the near-zero win rate across all agents and environments highlights a fundamental limitation: none of the agents learn the precise late-game navigation skills needed to fill the entire board. The reward signal for winning is too sparse for the agents to discover this behavior through random exploration alone.

References

Konda, V. R. and Tsitsiklis, J. N. Actor-critic algorithms. In *Advances in neural information processing systems*, pp. 1008–1014, 1999.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.

Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. MIT press, 2018.

Williams, R. J. Simple statistical gradient-following algo-

rithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.